

# Draft Hypercare

## PRD: Aplikasi Layanan Kesehatan

- 1. Visi Utama** Digitalisasi operasional medis harian. Integrasi data seluruh unit.
- 2. Fase 1: MVP Core** Pendaftaran rekam data pasien. Plasma antrian tampilkan nomor. Antrian realtime pakai Socket.io. Poliklinik kelola rekam medis. Farmasi proses resep dokter. Kasir terima pembayaran pasien.
- 3. Fase 2: Medis Lanjutan** IGD tangani kondisi darurat. Rawat Inap kelola kamar. Laboratorium catat hasil tes. Radiologi kelola citra medis. Fisioterapi atur jadwal sesi. Gizi pantau diet pasien.
- 4. Fase 3: Back Office** Pengadaan beli alat medis. Gudang pantau pergerakan stok. Keuangan catat arus kas. Akunting susun jurnal keuangan. CRM kirim pesan otomatis. Dashboard pantau metrik utama. Laporan cetak rekapitulasi data.
- 5. Spesifikasi Teknis** Backend pakai framework Laravel. Layanan pendukung pakai Node.js. Simpan data pakai PostgreSQL.

Rancang Entity-Relationship Diagram terlebih dahulu.

**Modul 1: Pendaftaran & Plasma** Catat data demografi pasien baru. Buat nomor rekam medis unik. Gunakan Node.js untuk sistem WebSocket. Plasma antrian wajib tampil responsif.

**Modul 2: Poliklinik** Dokter wajib input anamnesa pasien. Gunakan standar kode diagnosa ICD-10. Sistem buat resep obat elektronik. Kirim resep langsung ke farmasi.

**Modul 3: Farmasi** Apoteker harus validasi resep dokter. Sistem kurangi stok obat otomatis. Waspadai race condition saat update. Gunakan fitur transaction pada PostgreSQL.

**Modul 4: Kasir** Sistem gabungkan seluruh tagihan medis. Hitung total biaya secara presisi. Kasir cetak struk bukti pembayaran. Ubah status antrian menjadi selesai.

Hindari integer ID auto increment. Integer ID mudah dieksploitasi peretas. Gunakan tipe UUID tingkatkan keamanan.

Berikut struktur inti tabel MVP. Tabel Pasien simpan profil demografi. Tabel Kunjungan rekam antrian pasien. Tabel Anamnesa simpan diagnosa dokter. Tabel Resep catat kebutuhan obat. Tabel Tagihan rekap biaya akhir.

Jangan gabungkan tabel rekam medis. Satu kunjungan memiliki banyak tindakan. Normalisasi tabel tingkatkan performa query.

Skema statis mempersulit perubahan struktur. Manfaatkan tipe data JSONB PostgreSQL. JSONB membuat form sangat fleksibel. Gunakan transaction cegah anomali data.

Pilihan ORM (Eloquent) diterima.

Kelemahan ORM: masalah N+1 query. Performa turun pada query kompleks. Solusi: Wajib gunakan *Eager Loading*. Gunakan *raw query* untuk laporan.

Konfigurasi wajib Eloquent PostgreSQL: Gunakan trait `HasUuids` Laravel. Set properti `incrementing` menjadi `false`. Gunakan *casting* array untuk JSONB. Bungkus relasi dengan `DB::transaction()`.

Skema dasar pasien sering salah. Data medis pantang dihapus permanen. Gunakan fitur *soft deletes* Laravel. Tambahkan indeks pada kolom pencarian. Pencarian NIK wajib pakai indeks. Berikut kode *migration* tabel pasien.

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    public function up() {
        Schema::create('pasien', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->string('no_rm', 15)->unique();
            $table->string('nik', 16)->unique();
            $table->string('nama_lengkap', 150);
            $table->date('tanggal_lahir');
            $table->jsonb('riwayat_alergi')->nullable();
            $table->timestamps();
            $table->softDeletes();
        });
    }
};
```

Kesalahan umum: lupa melacak status. Plasma antrian butuh status realtime. Wajib buat kolom nomor antrian. Gunakan foreign key ID pasien. Indeks kolom status tingkatkan performa.

Berikut kode *migration* tabel kunjungan.

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
```

```

return new class extends Migration {
public function up() {
Schema::create('kunjungan', function (Blueprint $table) {
$table->uuid('id')->primary();
$table->foreignUuid('pasien_id')->constrained('pasien');
$table->string('kode_poli', 10)->index();
$table->integer('nomor_antrian');
$table->string('status', 20)->index();
$table->jsonb('tanda_vital')->nullable();
$table->timestamps();
$table->softDeletes();
});
}
};

```

Teks bebas mempersulit analisa data. Dokter sering menulis singkatan medis. Pisahkan data terstruktur dan teks. Wajib gunakan standar kode ICD-10. Gunakan JSONB untuk rekam detail. Berikut kode migration tabel anamnesa.

```

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
public function up() {
Schema::create('anamnesa', function (Blueprint $table) {
$table->uuid('id')->primary();
$table->foreignUuid('kunjungan_id')->constrained('kunjungan');
$table->string('kode_icd10', 10)->index();
$table->text('keluhan_utama');
$table->jsonb('pemeriksaan_fisik')->nullable();
$table->timestamps();
$table->softDeletes();
});
}
};

```

Jangan gabung tabel resep. Pisahkan induk dan detail obat. Penggabungan timbulkan redundansi data. Gunakan tabel induk untuk status. Gunakan detail untuk daftar obat. Validasi stok saat update status. Berikut skema tabel resep.

```

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

```

```

return new class extends Migration {
public function up() {
Schema::create('resep', function (Blueprint $table) {
$table->uuid('id')->primary();
$table->foreignUuid('kunjungan_id')->constrained('kunjungan');
$table->string('status', 20)->index();
$table->text('keterangan')->nullable();
$table->timestamps();
$table->softDeletes();
});
};

```

```

        Schema::create('resep_detail', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('resep_id')->constrained('resep')->
>cascadeOnDelete();
            $table->string('kode_obat', 20)->index();
            $table->integer('jumlah');
            $table->string('aturan_pakai', 100);
            $table->timestamps();
        });
    }
}

```

```
};
```

Mengacu harga master itu salah. Harga master pasti selalu berubah. Itu merusak riwayat tagihan lawas. Simpan nominal harga saat transaksi. Wajib gunakan tipe data decimal. Tipe float ciptakan selisih hitung. Pisahkan tabel tagihan dan rincian. Berikut skema tabel tagihan kasir.

```

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

```

```

return new class extends Migration {
public function up() {
Schema::create('tagihan', function (Blueprint $table) {
$table->uuid('id')->primary();
$table->foreignUuid('kunjungan_id')->constrained('kunjungan');
$table->decimal('total_biaya', 15, 2);
$table->string('status', 20)->index();
$table->timestamps();
$table->softDeletes();
});
};

```

```

Schema::create('tagihan_detail', function (Blueprint $table) {
    $table->uuid('id')->primary();
    $table->foreignUuid('tagihan_id')->constrained('tagihan')->cascadeOnDelete();
    $table->string('nama_item', 100);
    $table->decimal('harga_satuan', 15, 2);
    $table->integer('jumlah');
    $table->decimal('subtotal', 15, 2);
    $table->timestamps();
});
}

```

```
};
```

Polling database hancurkan performa server. Jangan panggil Node.js secara sinkron. Itu perlambat respon API Laravel. Gunakan Redis untuk perantara pesan. Laravel kirim event ke Redis. Node.js tangkap event dari Redis. Node.js *broadcast* ke plasma antrian.

Berikut kode *trigger* sisi Laravel.

```
use Illuminate\Support\Facades\Redis;
```

```

$data = [
    'nomor_antrian' => 12,
    'kode_poli' => 'UMUM'
];

```

```
Redis::publish('antrianchannel', json_encode($data));
```

Berikut kode *\_server* sisi Node.js.

JavaScript

```

const io = require('socket.io')(3000, {
  cors: { origin: "*" }
});
const Redis = require('ioredis');
const redis = new Redis();

redis.subscribe('antrian_channel');

redis.on('message', (channel, message) => {
  const data = JSON.parse(message);
  io.emit('update_plasma', data);
});

```

Framework berat turunkan performa klien. React berlebihan untuk plasma antrian. Plasma berjalan pada perangkat lawas. Gunakan Vanilla JavaScript saja. Kode jauh lebih ringan. Koneksi WebSocket butuh penanganan *reconnect*. Jaringan rumah sakit sering putus.

Berikut kode klien HTML sederhana.

## HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Plasma Antrian</title>
  <script src="https://cdn.socket.io/4.7.2/socket.io.min.js"></script>
</head>
<body>
  <h1 id="nomor">Menunggu...</h1>
  <h2 id="poli">--</h2>

  <script>
    const socket = io('http://localhost:3000', {
      reconnection: true,
      reconnectionAttempts: Infinity,
      reconnectionDelay: 1000,
    });

    const nomorEl = document.getElementById('nomor');
    const poliEl = document.getElementById('poli');

    socket.on('update_plasma', (data) => {
      nomorEl.innerText = data.nomor_antrian;
      poliEl.innerText = data.kode_poli;
    });

    socket.on('disconnect', () => {
      nomorEl.innerText = 'Terputus...';
    });
  </script>
</body>
</html>
```

Wajib gunakan GitHub simpan repositori. Bangun alur CI/CD otomatis.

Validasi frontend saja itu fatal. Peretas mudah memanipulasi data klien. Jangan pernah percaya input pengguna. Wajib gunakan Form Request Laravel. Controller gemuk adalah pola buruk. Pisahkan logika bisnis ke Service. Ini mempermudah proses pengujian unit.

Berikut kode Form Request validasi.

PHP

```
namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreAnamnesaRequest extends FormRequest {
    public function authorize() {
        return true;
    }

    public function rules() {
        return [
            'kunjungan_id' => 'required|uuid|exists:kunjungan,id',
            'kode_icd10' => 'required|string|max:10',
            'keluhan_utama' => 'required|string',
            'pemeriksaan_fisik' => 'nullable|array'
        ];
    }
}
```

Berikut kode Controller yang bersih.

PHP

```
namespace App\Http\Controllers;

use App\Http\Requests\StoreAnamnesaRequest;
use App\Services\AnamnesaService;

class AnamnesaController extends Controller {
    public function store(StoreAnamnesaRequest $request, AnamnesaService $service) {
        $data = $service->simpanAnamnesa($request->validated());

        return response()->json([
            'status' => 'sukses',
            'data' => $data
        ], 201);
    }
}
```

Lupa update status itu fatal. Data antrian menjadi sangat kacau. Wajib gunakan *database transaction*. Satu gagal, batalkan semua proses. Berikut kode *Service Class* poliklinik.

PHP

```
namespace App\Services;

use App\Models\Anamnesa;
use App\Models\Kunjungan;
use Illuminate\Support\Facades\DB;

class AnamnesaService {
    public function simpanAnamnesa(array $data) {
        return DB::transaction(function () use ($data) {
            $anamnesa = Anamnesa::create($data);

            Kunjungan::where('id', $data['kunjungan_id'])
                ->update(['status' => 'DIPERIKSA']);

            return $anamnesa;
        });
    }
}
```

Abaikan race condition sangat fatal. Dua apoteker kurangi stok bersamaan. Stok obat pasti menjadi minus. Validasi stok biasa sangat cacat. Wajib gunakan *pessimistic locking* database. Gunakan `lockForUpdate()` pada Eloquent. Ini kunci baris saat transaksi. Berikut kode *Service* modul farmasi.

PHP

```
namespace App\Services;

use App\Models\Obat;
use App\Models\ResepDetail;
use Illuminate\Support\Facades\DB;
use Exception;

class FarmasiService {
    public function prosesResep(array $data) {
        return DB::transaction(function () use ($data) {
            $obat = Obat::where('kode_obat', $data['kode_obat'])
                ->lockForUpdate()
                ->first();
```

```

        if ($obat->stok < $data['jumlah']) {
            throw new Exception('Stok tidak cukup');
        }

        $obat->decrement('stok', $data['jumlah']);

        return ResepDetail::create($data);
    });
}

```

Pembayaran ganda sangat berbahaya. Pasien bisa bayar dua kali. Validasi status saja tidak cukup. Wajib gunakan *pessimistic locking* lagi. Kunci baris tagihan saat proses. Ubah status kunjungan menjadi selesai. Berikut kode *Service* kasir.

## PHP

```

namespace App\Services;

use App\Models\Tagihan;
use App\Models\Kunjungan;
use Illuminate\Support\Facades\DB;
use Exception;

class KasirService {
    public function bayarTagihan(string $tagihanId) {
        return DB::transaction(function () use ($tagihanId) {
            $tagihan = Tagihan::where('id', $tagihanId)
                ->lockForUpdate()
                ->first();

            if ($tagihan->status === 'LUNAS') {
                throw new Exception('Tagihan sudah lunas');
            }

            $tagihan->update(['status' => 'LUNAS']);

            Kunjungan::where('id', $tagihan->kunjungan_id)
                ->update(['status' => 'SELESAI']);

            return $tagihan;
        });
    }
}

```

Arsitektur SPA murni perlambat rilis. Pembuatan API terpisah buang waktu. Itu kelemahan utama developer. Gunakan pendekatan monolith Inertia.js.

Laravel kendalikan seluruh sistem routing. React bangun komponen antarmuka interaktif. Aplikasi tetap terasa seperti SPA. Waktu pengerjaan berkurang sangat drastis.

Penumpukan komponen hancurkan skalabilitas kode. Pisahkan direktori menurut modul bisnis. Direktori Pendaftaran simpan halaman registrasi. Direktori Poliklinik simpan tampilan dokter. Direktori Farmasi simpan tampilan apotek. Direktori UI simpan komponen global.

Hindari state management skala global. Redux sangat perumit arsitektur aplikasi. Manfaatkan sistem prop komponen React. Controller Laravel suplai data langsung.

Otentikasi dasar aplikasi sangat berbahaya. Satu akun akses semua modul. Ini pelanggaran fatal data pasien. Wajib terapkan akses berbasis peran. Gunakan pustaka Spatie Laravel Permission. Pisahkan hak akses tiap profesi. Dokter dilarang masuk sistem kasir. Kasir dilarang lihat rekam medis.

Tabel pengguna butuh relasi peran. Kode penetapan hak akses berikut.

PHP

```
use Spatie\Permission\Models\Role;
use Spatie\Permission\Models\Permission;

$roleDokter = Role::create(['name' => 'dokter']);
$roleKasir = Role::create(['name' => 'kasir']);

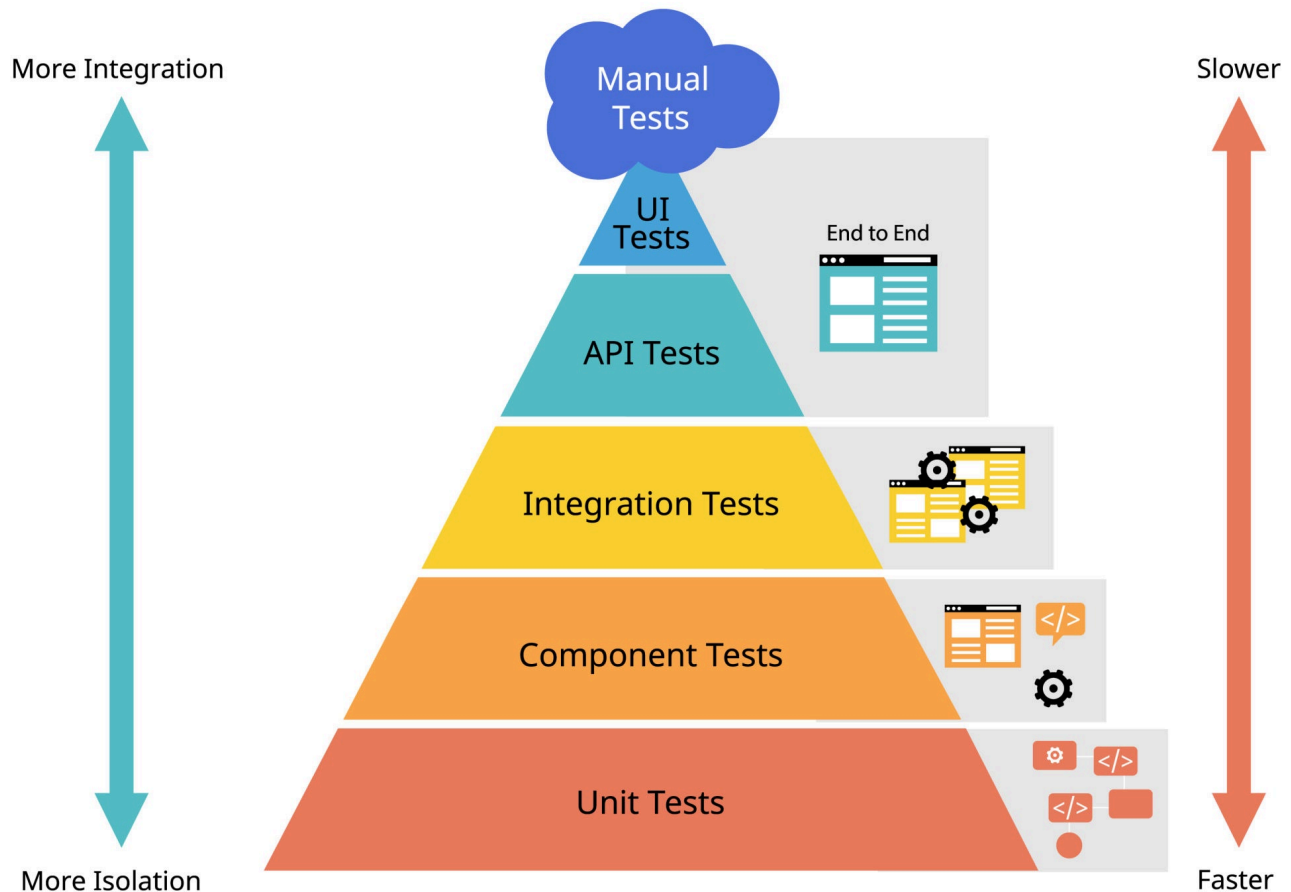
$izinMedis = Permission::create(['name' => 'akses medis']);
$izinBayar = Permission::create(['name' => 'akses bayar']);

$roleDokter->givePermissionTo($izinMedis);
$roleKasir->givePermissionTo($izinBayar);

$user->assignRole('dokter');
```

Deploy manual sangat berisiko tinggi. Downtime hancurkan operasional rumah sakit. Otomatisasi alur kerja pakai GitHub. Gunakan Docker bungkus seluruh lingkungan. Wajib pisahkan server database produksi. Satu server tumbang, data aman. Terapkan replikasi PostgreSQL tingkat lanjut. Gunakan VPS isolasi jaringan server. Backup otomatis wajib berjalan harian. Abaikan backup berarti bunuh diri.

Pengujian manual buang banyak waktu. Manusia pasti lewatkan banyak kutu. Wajib terapkan piramida otomatisasi pengujian.



## Piscine

Perbanyak pengujian unit level dasar. Uji logika kelas Service aplikasi. Pengujian integrasi periksa interaksi database. Pengujian ujung validasi alur pengguna. Gunakan kerangka Pest untuk Laravel. Berikut contoh uji logika kasir.

## PHP

```
test('kasir lunas ubah status kunjungan', function () {  
    $tagihan = Tagihan::factory()->create(['status' => 'BELUM']);  
  
    $service = new KasirService();  
    $service->bayarTagihan($tagihan->id);  
  
    expect($tagihan->fresh()->status)->toBe('LUNAS');  
});
```

Jalankan pengujian pakai GitHub Actions. Blokir rilis jika pengujian gagal.

Dokumentasi manual pasti cepat usang. Tim *frontend* pasti sangat kebingungan. Hindari dokumen format teks biasa. Wajib gunakan standar spesifikasi OpenAPI. Gunakan pustaka Scribe untuk Laravel. Scribe baca anotasi blok komentar. Dokumentasi selalu akurat dan sinkron. Berikut contoh blok anotasi kontroler.

PHP

```
/**
 * @group Pendaftaran Pasien
 *
 * Simpan data rekam medis baru.
 *
 * @bodyParam nik string required NIK KTP pasien. Example: 327301...
 * @bodyParam nama_lengkap string required Nama lengkap pasien.
 * @response 201 {"status": "sukses", "data": {...}}
 */
public function store(Request $request) {
    // Logika simpan data
}
```

Membaca file log buang waktu. Tim pasti lewatkan galat kritis. Wajib gunakan pelacak galat otomatis. Gunakan Sentry pantau aplikasi realtime. Sentry rekam detail tumpukan eksekusi. Galat langsung masuk grup Telegram. Sistem bantu identifikasi akar masalah. Berikut kode integrasi Sentry Laravel.

PHP

```
public function register() {
    $this->reportable(function (Throwable $e) {
        \Sentry\Laravel\Integration::captureUnhandledException($e);
    });
}
```